

VPEM Graph

Vulnerability Prioritization and Exposure Management

Cybersecurity Use Cases: Evolving from reactive patching to risk-based strategy

Based on: [pedroleitao-neo4j/cyber-vpem](https://github.com/pedroleitao-neo4j/cyber-vpem)

The Problem: Alert Fatigue

Modern organizations are overwhelmed by thousands of software flaws.

- **The spreadsheet trap:** Treating a critical flaw on an isolated server the same as one on an internet-facing gateway.
- **Reactive vs Proactive:** Wasting resources on theoretical risks instead of real-world threats.
- **Lack of Context:** Severity scores (CVSS) don't account for your specific environment.

What is VPEM?

It shifts the focus from "What is broken?" to "What could actually hurt us right now?"

- **Exposure:** Is the system public-facing or internal?
- **Threat Intel:** Is there an active exploit in the wild (CISA KEV)?
- **Criticality:** Is this a test server or a production "Crown Jewel" database?

Core Architecture

The essentials only

1. **Ingest NVD + KEV ([loader.ipynb](#)):** Batch `UNWIND` + `MERGE` from local [cvelistV5](#) and enrich with CISA KEV.
2. **Run VPEM queries ([vpem.ipynb](#)):** Reachability, Blast Radius, Remediation ROI, Contextual Risk Scoring.
3. **Explore extras ([other.ipynb](#)):** CVE/CWE analyses and risk concentration patterns.

You can [use your own data](#), just swap out the synthetic infra with real asset inventory.

Data Integration Layers

To build a holistic view, three distinct data domains are integrated:

Layer	Sources	Key Entities
Organizational Context	Synthetic infra in repo	Apps, Build Artifacts, Compute, Endpoints, Identities, IAM Policies, Cloud Services
Vulnerability Intel	Local NVD repo (cvelistV5)	CVEs, CVSS, CWEs, Vendors, Products
Threat Intelligence	CISA KEV feed	Known-exploited flag, due dates, catalog linkage

Ingestion Snapshot (NVD + KEV)

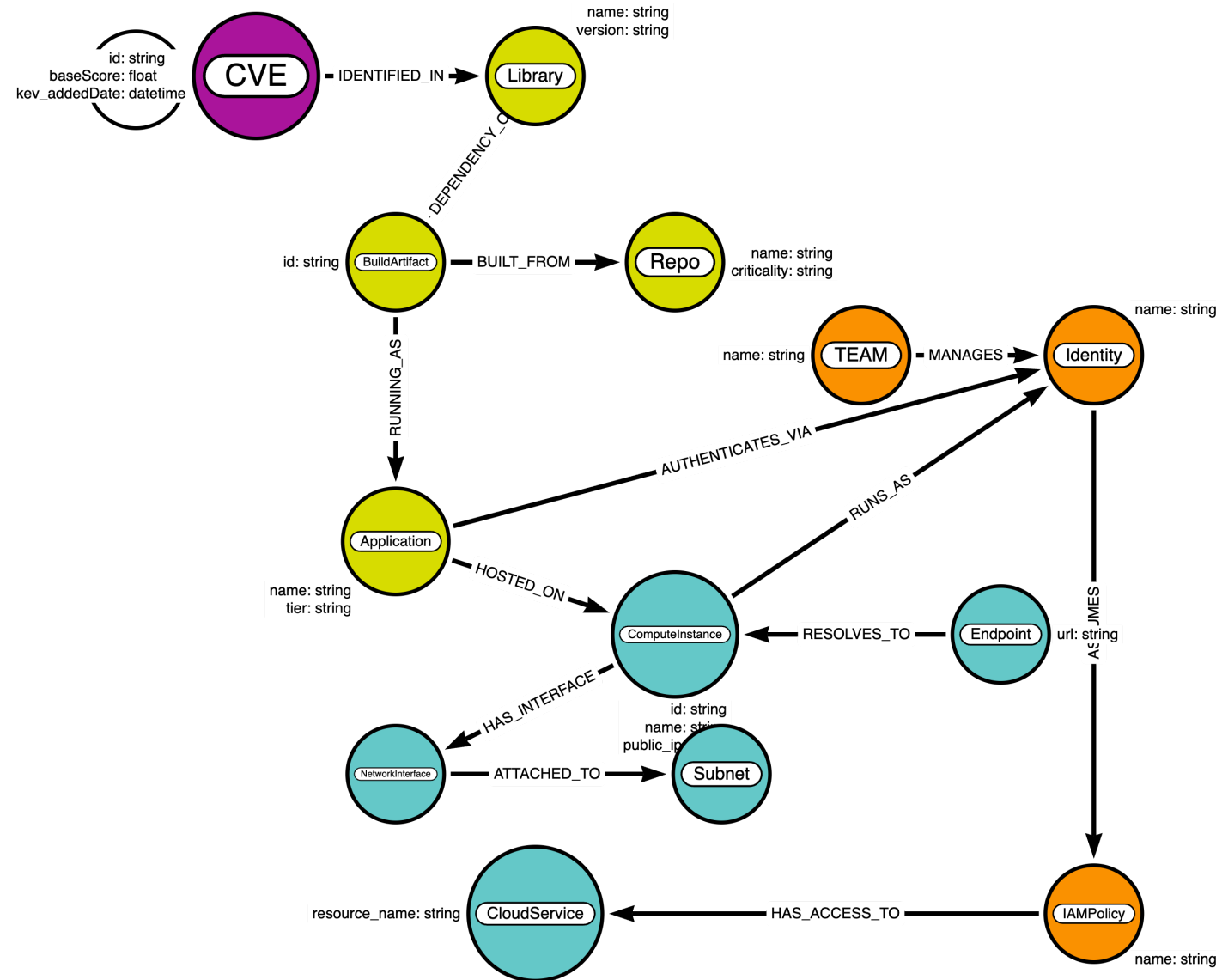
What we ingest and how

- **NVD CVEs:** Read local JSONs under [cvelistV5/cves](#); ingest only PUBLISHED entries; consolidate CNA + ADP metrics; capture CVE/CWE and link CVEs to `Vendor → Product` via `AFFECTS`.
- **Performance & Idempotency:** Use batched, transactional loads (UNWIND/MERGE semantics) and create uniqueness constraints for `CVE.id`, `CWE.id`, `Vendor.name`, `Product.name`.
- **KEV Enrichment:** Fetch the official CISA KEV feed; flag CVEs as known exploited; add due/added dates; and link each CVE to a `catalog` node for traceability.

The Resulting Schema

The graph connects **code**, **infrastructure**, and **identity** in real-time.

The extended schema captures deployment, usage, and vulnerability identification.



The Graph Advantage

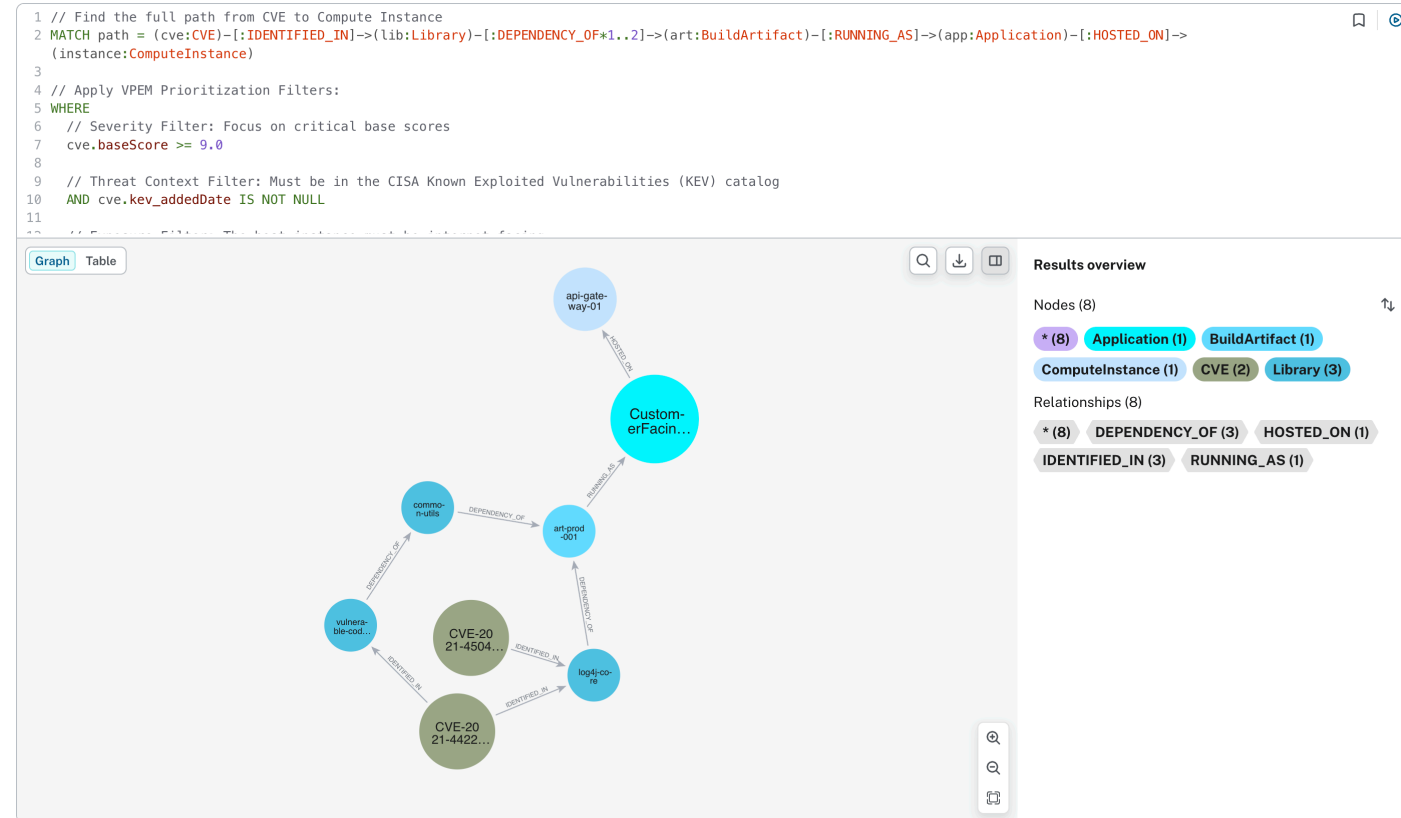
Why use Neo4j for Vulnerability Management?

- **Real-world Reachability:** Is the vulnerable library actually reachable from an `Endpoint` ?
- **Impact Analysis:** If this server is compromised, which "Crown Jewel" assets (PII databases) are at risk via its Identity?
- **Efficiency:** Find the "Chokepoint" - the one library update that fixes the most reachable risk.

Visualizing Reachability

Vulnerability to Compute Instance Reachability

- **Attack Path Analysis:** Identifying the direct path from a CVE to an internet-facing host.
- **Blast Radius:** Mapping what a compromised identity can access.



Reachability Example (Log4Shell)

Is a known CVE exposed to the internet?

```
MATCH (v:CVE)-[:IDENTIFIED_IN]->(l:Library)
      -[:DEPENDENCY_OF]->(:BuildArtifact)
      -[:RUNNING_AS]->(app:Application)
      -[:HOSTED_ON]->(i:ComputeInstance)
WHERE v.id = 'CVE-2021-44228' AND EXISTS { (:Endpoint)-[:RESOLVES_TO]->(i) }
RETURN app.name AS Application, i.public_ip AS Public_IP, v.id AS CVE
```

Why it matters: filters out non-exploitable findings by requiring an internet-facing

Endpoint → ComputeInstance link.

Impact Example (Blast Radius)

What could the compromised app access?

```
MATCH (v:CVE)-[:IDENTIFIED_IN]->(:Library)
      -[:DEPENDENCY_OF]->(:BuildArtifact)
      -[:RUNNING_AS]->(app:Application)
MATCH (app)-[:AUTHENTICATES_VIA]->(:Identity)
      -[:ASSUMES]->(:IAMPolicy)
      -[:HAS_ACCESS_TO]->(cs:CloudService)
RETURN v.id AS CVE, app.name AS Application, cs.resource_name AS Resource
```

Connects software risk to cloud data access (e.g., PII bucket exposure).

Remediation ROI Example

Which repo fix removes the most exposed risk?

```
MATCH (v:CVE)-[:IDENTIFIED_IN]->(:Library)
      -[:DEPENDENCY_OF]->(:BuildArtifact)
      -[:BUILT_FROM]->(r:Repo)
MATCH (r)<-[:BUILT_FROM]-(:BuildArtifact)
      -[:RUNNING_AS]-(:Application)
      -[:HOSTED_ON]->(i:ComputeInstance)
WHERE i.public_ip IS NOT NULL
RETURN r.name AS Repo,
       COUNT(DISTINCT v) AS Total_Vulns,
       COUNT(DISTINCT i) AS Exposed_Instances
ORDER BY Total_Vulns DESC
```

Targets high-ROI dependency updates at the source (repository ownership).

Contextual Risk Scoring

Dynamic score = CVSS × Reachability × Impact

```
MATCH (v:CVE)-[:IDENTIFIED_IN]->(:Library)
      -[:DEPENDENCY_OF]->(:BuildArtifact)
      -[:RUNNING_AS]->(app:Application)
      -[:HOSTED_ON]->(ins:ComputeInstance)
WITH v, app, ins,
      CASE WHEN ins.public_ip IS NOT NULL THEN 1.5 ELSE 1.0 END AS reach_mult
OPTIONAL MATCH (ins)-[:RUNS_AS]->(:Identity)
      -[:ASSUMES]->(:IAMPolicy)
      -[:HAS_ACCESS_TO]->(:CloudService)
WITH v, app, reach_mult,
      CASE WHEN count(*) > 0 THEN 2.0 ELSE 1.0 END AS impact_mult
RETURN v.id AS CVE,
       app.name AS Application,
       v.baseScore AS Base_CVSS,
       reach_mult AS Reach_Mult,
       impact_mult AS Impact_Mult,
       round(v.baseScore * reach_mult * impact_mult, 2) AS Final_Risk
ORDER BY Final_Risk DESC
```

Prioritizes internet-exposed paths with sensitive-data access to the top.

Outward System Integration

The VPEM graph acts as a **central nervous system** for security data:

- **SecOps (SOAR):** Contextual alert enrichment and automated containment.
- **DevOps (CI/CD):** "Policy as Code" guardrails to fail builds that create lateral movement paths.
- **GRC & Compliance:** Strategic risk metrics and CISA KEV remediation tracking.

Questions?

GitHub: [pedroleitao-neo4j/cyber-vpem](https://github.com/pedroleitao-neo4j/cyber-vpem)